KNOWLEDGE INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Approved by AICTE, New Delhi and Affiliated to Anna University, Chennai

Kakapalayam (Po), Salem – 637 504

RECORD NOTE BOOK

REG. NO.

Certified that this is the Bonafide record of work done by Selvan/Selvi …….…… of the

IV Semester _____Branch during the year **2025 - 26** in the **BE23AD531 – FOUNDATIONS OF DATA ENGINEERING** Laboratory.

Staff – Incharge **Head of the Department**

_____

Internal Examiner

CONTENTS

EX. NO.	DATE	NAME OF THE EXPERIMENT	PAGE NO.	MARKS	SIGNATURE
1.		Explore the Data Lifecycle and Ecosystem			
2.		Design a Simple Database and Query Using SQL			
3.		Extract, Transform, and Load (ETL) a Dataset			
4.		Perform Batch Processing Using PySpark			
5.		Simulate a Streaming Pipeline Using Apache Kafka			
6.		Orchestrate a Workflow Using Apache Airflow			

KNOWLEDGE INSTITUTE OF TECHNOLOGY, SALEM – 637504

(Affiliated to Anna University, Chennai)
(Accredited by NAAC)

Department of Artificial Intelligence and Data Science

Vision:

To create globally competent industry-ready graduates in the field of Artificial Intelligence and Data Science to solve societal problems through cutting-edge research and entrepreneurship skills with values and ethics.

Mission:

Mission 1: To inculcate innovation and creativity through experiential learning with the modern infrastructure and technologies

Mission 2: To collaborate with the industry for enhancing the students' research ability on cutting edge technologies of AI and Data Science

Mission 3: To develop industry-ready professionals with an ethical attitude and entrepreneurship skills

Programme Educational Objectives

The graduates of Artificial Intelligence and Data Science will be able to

PEO1: Perform data processing, analysis and visualization in real time applications for better prediction and data-driven decision making

PEO2: Enable multitasking for existing resources and execute complex tasks using Artificial Intelligence

PEO3: Carry out fundamental research to cater the critical needs of the society through cutting edge technologies of AI

Program Specific Objectives (PSOs):

The graduates of Artificial Intelligence and Data Science will be able to

PSO1: Apply the concepts of Machine Learning and Data Science to solve the real time business problems

PSO2: Exhibit their professional skills in team building, leadership, communication, values and ethics

PSO3: Build a suitable model to assist business analytics and helps solving business problems

PROGRAM OUTCOMES (POs)

1.Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2.Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3.Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4.Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5.Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6.The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice**.**

7.Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8.Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9.Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10.Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11.Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one’s own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12.Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

BE23AD404 - DATA ENGINEERING TOOLS AND TECHNIQUES LABORATORY

OBJECTIVES:

- To introduce the foundational concepts of data engineering and its role in modern data ecosystems.
- To enable students to understand data modeling, storage, and processing techniques.
- To provide hands-on knowledge of ETL pipelines and workflow orchestration.
- To explore modern cloud-based tools and frameworks for data engineering.
- To prepare students to design and implement end-to-end data pipelines and workflows.

LIST OF EXPERIMENTS:

1. Explore the Data Lifecycle and Ecosystem
2. Design a Simple Database and Query Using SQL
3. Extract, Transform, and Load (ETL) a Dataset
4. Perform Batch Processing Using PySpark
5. Simulate a Streaming Pipeline Using Apache Kafka
6. Orchestrate a Workflow Using Apache Airflow

OUTCOMES:

at the end of this course, the students will be able to:

CO1: Understand the role and significance of data engineering in the data ecosystem.

CO2: Design and implement data models and ETL pipelines for relational and NoSQL databases.

CO3: Apply data storage and processing frameworks for batch and real-time data.

CO4: Build workflow orchestration pipelines and explore cloud-based data engineering solutions.

CO5: Demonstrate the ability to design end-to-end data pipelines.

CONTENT BEYOND SYLLABUS

1. Implementing DAG Versioning and CI/CD for Airflow Pipelines
2. Implementing Delta Lake for ACID Transactions in Spark

Total: 30 Periods

EX.NO:1	EXPLORATION OF THE DATA LIFECYCLE AND ECOSYSTEM

AIM

To understand and demonstrate the stages of the data lifecycle and the components of the data ecosystem through a simple Python-based simulation.

PROCEDURE

1. Start the Python environment (e.g., Jupyter Notebook, VS Code, or any Python IDE).
2. Import required libraries such as pandas, numpy, and matplotlib for data handling and visualization.
3. Generate sample data representing real-world input (e.g., temperature and humidity readings).
4. Create a DataFrame using the sample data to simulate the raw data collection stage.
5. Clean the data by handling missing values using methods like forward fill (fillna(method='ffill')).
6. Store the cleaned data by saving it into a .csv file to simulate the data storage phase.
7. Analyze the data by calculating basic statistics like average temperature and humidity.
8. Visualize the data using a line graph to show trends in temperature and humidity over time.
9. Simulate data archival by keeping the cleaned data file and printing a message to indicate archival or deletion.
10. Observe and interpret the outputs and visualizations to understand how each phase reflects a part of the data lifecycle.

PROGRAM

```
# Import necessary libraries

import pandas as pd

import matplotlib.pyplot as plt

import numpy as np

# Step 1: Data Generation

print("Step 1: Generating sample data")

data = {

    'ID': range(1, 11),

    'Temperature': [22.5, 23.0, np.nan, 21.0, 22.0, 25.0, 24.5, np.nan, 23.5, 22.5],

    'Humidity': [30, 35, 32, 31, np.nan, 33, 30, 29, np.nan, 34]

}

df = pd.DataFrame(data)

print(df)

# Step 2: Data Cleaning

print("\n Step 2: Cleaning Data (Handling missing values)")

df_cleaned = df.fillna(method='ffill')

print(df_cleaned)

# Step 3: Data Storage (Simulated as saving to CSV)

print("\n Step 3: Saving cleaned data to 'cleaned_data.csv'")

df_cleaned.to_csv("cleaned_data.csv", index=False)

# Step 4: Data Analysis

print("\n Step 4: Analyzing Data (Mean Temperature & Humidity)")

mean_temp = df_cleaned['Temperature'].mean()
```

```
mean_hum = df_cleaned['Humidity'].mean()

print(f"Average Temperature: {mean_temp:.2f} °C")

print(f"Average Humidity: {mean_hum:.2f} %")

# Step 5: Data Visualization

print("\n Step 5: Visualizing Temperature and Humidity")

plt.figure(figsize=(10, 4))

plt.plot(df_cleaned['ID'], df_cleaned['Temperature'], marker='o', label='Temperature (°C)')

plt.plot(df_cleaned['ID'], df_cleaned['Humidity'], marker='x', label='Humidity (%)')

plt.title("Temperature and Humidity Over Time")

plt.xlabel("Record ID")

plt.ylabel("Values")

plt.legend()

plt.grid(True)

plt.tight_layout()

plt.show()

# Step 6: Archival / Deletion

print("\n Step 6: Archiving or Deleting Data (Simulated)")

print("Cleaned data archived in 'cleaned_data.csv'.")
```

OUTPUT

```
Step 1: Generating sample data
     ID Temperature    Humidity
      1    22.5  23.0      30.0
      2    23.0  23.0      32.0
      3    21.0  22.0      31.0
      4    22.0  22.0      31.0
      5    25.0  25.0      33.0
      6    24.5  24.5      30.0
      7    24.5  24.5      30.0
      8    23.5  23.5      29.0
      9    22.5  22.5      34.0

Step 2: Cleaning Data (Handling missing values)
     ID Temperature    Humidity
      1    22.5  23.0      30.0
      2    23.0  23.0      32.0
      3    21.0  22.0      31.0
      4    22.0  22.0      31.0
      5    25.0  25.0      33.0
      6    24.5  24.5      30.0
      7    24.5  24.5      29.0
      8    22.5  22.5      34.0

Step 3: Saving cleaned data to 'cleaned_data.csv'
Step 4: Visualizing Data (Mean Temperature & Humidty
        Average Temperature: 23.35 °C
        Average Humidity: 31.40 %
```

DEPARTMENT OF AI&DS		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT

The data lifecycle was successfully simulated using Python by generating, cleaning, analyzing, visualizing, and storing sample sensor data.

EX.NO:2	DESIGN A SIMPLE DATABASE AND QUERY USING SQL

AIM

To design a simple relational database, insert sample data, and execute SQL queries to retrieve meaningful information.

PROCEDURE

1. Design the schema with appropriate tables and data types.
2. Create the database and tables using SQL `CREATE` statements.
3. Insert sample data using the `INSERT INTO` statement.
4. Query the data using `SELECT`, `WHERE`, `JOIN`, `GROUP BY`, and `ORDER BY`.
5. Test the results and validate the output for correctness.

PROGRAM

Step 1: Create the database

```
CREATE DATABASE CompanyDB;
USE CompanyDB;
```

Step 2: Create tables

```
CREATE TABLE Employees (
   EmpID INT PRIMARY KEY,
   Name VARCHAR(50),
   DepartmentID INT,
   Salary DECIMAL(10,2)
);
CREATE TABLE Departments (
   DepartmentID INT PRIMARY KEY,
   DepartmentName VARCHAR(50)
);
```

Step 3: Insert sample data

```
INSERT INTO Departments VALUES (1, 'HR'), (2, 'IT'), (3, 'Finance');
INSERT INTO Employees VALUES
```

(101, 'Alice', 2, 70000.00),

(102, 'Bob', 1, 50000.00),

(103, 'Charlie', 2, 65000.00),

(104, 'David', 3, 60000.00),

(105, 'Eva', 1, 52000.00);

Step 4: Query - List employees with department names and salaries > 55000

```
SELECT e.Name, d.DepartmentName, e.Salary
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE e.Salary > 55000
ORDER BY e.Salary DESC;
```

OUTPUT

```
+------------+-----------------+------------+
| Name       | DepartmentName  | Salary     |
+------------+-----------------+------------+
| Alice      | IT              |   70000.00 |
| Charlie    | IT              |   65000.00 |
| David      | Finance         |   60000.00 |
+------------+-----------------+------------+
3 rows in set (0.02 sec)
```

DEPARTMENT OF AI&DS		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT

A simple database with two related tables was successfully created, populated, and queried to retrieve employee names with their department and salary above 55,000.

EX.NO:3	EXTRACT, TRANSFORM, AND LOAD (ETL) A DATASET

AIM

To perform ETL process on real-time data using Python by cleaning, transforming, and enriching the data before saving it in a structured format.

PROCEDURE

1. Start your Python environment (such as Jupyter Notebook or VS Code).
2. Import the required libraries: pandas and numpy.
3. Extract: Simulate raw customer sales data using a Python dictionary and convert it into a DataFrame.
4. Transform the data:
    - Fill missing values in CustomerName, PurchaseAmount, PurchaseDate, and City.
    - Format strings in CustomerName and City columns to proper case.
    - Convert PurchaseDate to datetime format.
    - Add a new derived column PurchaseCategory based on the purchase amount using binning.
5. Load: Save the transformed dataset into a new CSV file named cleaned_sales_data.csv.
6. Verify: Print the final DataFrame and confirm successful ETL completion.

PROGRAM

```
import pandas as pd

import numpy as np

# Step 1: Extract - Simulate reading from a raw sales dataset

print("Extracting sales data...")

raw_sales_data = {

    'CustomerID': [101, 102, 103, 104, 105, 106],

    'CustomerName': ['Alice', 'Bob', 'Charlie', 'David', np.nan, 'Frank'],

    'PurchaseAmount': [250.50, 300.00, np.nan, 150.00, 200.00, 400.00],

    'PurchaseDate': ['2024-01-15', '2024-01-17', '2024-01-19', None, '2024-01-22', '2024-01-25'],

    'City': ['New York', 'Chicago', 'Los Angeles', 'Houston', np.nan, 'Seattle']

}

df = pd.DataFrame(raw_sales_data)

print(df)

# Step 2: Transform - Clean and enrich the data

print("\n Transforming data...")

# Fill missing values

df['CustomerName'].fillna('Unknown', inplace=True)

df['PurchaseAmount'].fillna(df['PurchaseAmount'].mean(), inplace=True)

df['PurchaseDate'].fillna('2024-01-01', inplace=True)

df['City'].fillna('Unknown', inplace=True)

# Format text columns

df['CustomerName'] = df['CustomerName'].str.title()
```

```
df['City'] = df['City'].str.title()


# Convert date column to datetime

df['PurchaseDate'] = pd.to_datetime(df['PurchaseDate'])

# Add a new column: Purchase Category

df['PurchaseCategory'] = pd.cut(df['PurchaseAmount'],

                                bins=[0, 199, 299, 399, np.inf],

                                labels=['Low', 'Medium', 'High', 'Very High'])

print(df)

# Step 3: Load - Save the transformed data

print("\n Loading data into 'cleaned_sales_data.csv'")

df.to_csv("cleaned_sales_data.csv", index=False)

# Step 4: Confirm completion

print("\n ETL process for customer sales data completed successfully.")
```

OUTPUT

```
Extracting Sales data...
Step 2: Extract - Simulate readeing from a
        raw sales dataset
CustomerID CustomerAmountt PurchaseDate       City
       101            250.0 2024-01-15     New York
       102            300.0 2024-01-17      Chicago
       103            260.0 2024-01-19  Los Angeles
       105            NaN   2024-01-22      Houston
    Unknow            200.0 2024-01-22     Unknown
     Frank            400.0 2024-01-25      Seattle
Step 2: Transform - Clean and enrich the data
        N Tranforming data...
CustomerID CustomerAmountt PurchaseDate PurchaseCategory
       101 Alice    250.50 2024-01-15   Medium
       102 Bob      300.00 2024-01-17   High
       103 Charlie  260.10 2024-01-19   Medium
     David David    150.00 2024-01-01   Low
   Unknown Unknown  200.00 2024-01-22   Low
     Frank Frank    400.00 2024-01-25   Very High
Step 3: Load - Save the 'cleaned_sales_data.csv'
Step 4: Confirm completion
        ETL process for customer sales data
        completed successfully.
```

DEPARTMENT OF AI&DS		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT

The ETL process on customer sales data was successfully completed by extracting, transforming, and loading the dataset into a clean, structured CSV file using Python.

EX.NO:4	PERFORM BATCH PROCESSING USING PYSPARK

AIM

To perform batch processing on large datasets using PySpark with data transformation, partitioning, and error handling.

PROCEDURE

1. **Set Up the Environment:**
    - Install PySpark and required libraries (pyspark, requests, logging).
    - Prepare the input CSV file (large_data.csv) with at least col1 and col2 columns.
2. **Configure Logging:**
    - Initialize a logger to capture info and error messages during execution.
3. **Initialize Spark Session:**
    - Create a SparkSession with necessary configurations such as memory settings and number of shuffle partitions.
4. **Read Input Data:**
    - Load the CSV data with schema inference and header recognition.
    - Perform schema validation to ensure required columns (col1, col2) are present.
5. **Optimize Partitioning:**
    - Dynamically adjust the number of partitions for better performance during processing.
6. **Process Each Partition:**
    - Use foreachPartition() to process each batch of data.
    - Inside process_batch():
        - Convert iterator to DataFrame.
        - Apply transformations: double col1, filter rows where col2 > 50.
        - (Optional) Simulate an API call per row using requests.post().
        - Save the processed batch to local disk or cloud storage (like AWS S3).
7. **Error Handling:**
    - Wrap processing in a try-except block to handle errors gracefully.
    - Log any failures for further analysis or retry mechanisms.
8. **Stop Spark Session:**
    - Ensure the Spark session is stopped after processing completes.

PROGRAM

```
# batch_processing.py

from pyspark.sql import SparkSession

from pyspark.sql.types import IntegerType

import logging

import sys


# Configure logging

logging.basicConfig(level=logging.INFO)

logger = logging.getLogger(__name__)


def process_batch(rows_iterator):

    """

    Processes a batch of data with error handling and API simulation

    """

    from pyspark.sql import SparkSession

    import requests

    import json


    try:

        # Create local Spark session for this partition

        local_spark = SparkSession.builder.getOrCreate()


        # Convert iterator to list and create DataFrame

        rows = list(rows_iterator)
```

```
        if not rows:
            return

        batch_df = local_spark.createDataFrame(rows)

        # Example transformation pipeline
        transformed_df = batch_df \
            .withColumn("col1", batch_df["col1"] * 2) \
            .filter(batch_df["col2"] > 50)

        # Simulate API call (replace with actual endpoint)
        # api_endpoint = "https://your-api.com/data"
        # for row in transformed_df.collect():
        #     response = requests.post(api_endpoint, json=row.asDict())
        #     if response.status_code != 200:
        #         logger.error(f"API call failed for row: {row}")

        # Write processed batch (AWS S3 or local)
        transformed_df.write.mode("append").format("csv") \
            .option("header", "true") \
            .save("output/")  # For AWS: "s3://your-bucket/output/"

        logger.info(f"Processed batch with {transformed_df.count()} records")

    except Exception as e:
        logger.error(f"Batch processing failed: {str(e)}")
```

```
        # Implement retry logic or dead-letter queue here


def main(input_path):
    spark = SparkSession.builder \
        .appName("PySparkBatchProcessing") \
        .config("spark.sql.shuffle.partitions", "10") \
        .config("spark.executor.memory", "4g") \
        .master("local[*]") \  # For cluster: "yarn" or "spark://<master>:7077"
        .getOrCreate()

    try:
        # Read input data (local or S3)
        df = spark.read.csv(input_path,
                            header=True,
                            inferSchema=True,
                            enforceSchema=True)

        # Validate schema
        if not set(df.columns) >= {"col1", "col2"}:
            raise ValueError("Invalid schema - missing required columns")

        logger.info(f"Input data schema: {df.schema.json()}")
        logger.info(f"Total records: {df.count()}")

        # Optimize partitioning
        ideal_partitions = max(df.rdd.getNumPartitions(), 10)
```

```
        optimized_df = df.repartition(ideal_partitions)

        # Process batches using Spark partitions
        optimized_df.rdd.foreachPartition(process_batch)

        logger.info("Batch processing completed successfully")

    finally:
        spark.stop()

if __name__ == "__main__":
    # For local: "large_data.csv"
    # For AWS: "s3://your-bucket/input/large_data.csv"
    input_path = sys.argv[1] if len(sys.argv) > 1 else "large_data.csv"
    main(input_path)
```

OUTPUT

```
Extracting input data...
INFO:root:Input["filed5:"coıllı";
"IntegerType","nunluable":true,"]]
Total records: 1000
INFO:root Processed batch with 150
records  Processed batch with 180
records  Processed batch with 170
records  Processed batch with 160
records  Processed batch with 170
records  Processed batch with 170
Batch processing completed
successfully
```

DEPARTMENT OF AI&DS		
AIM & ALGORITHM	20	
PROGRAM	30	
EXECUTION & OUTPUT	30	
OBSERVATION	10	
VIVA VOCE	10	
TOTAL	100	

RESULT

The batch data was successfully processed using PySpark with transformations, partition-level handling, and output saved to the specified location.

EX.NO:5	SIMULATE A STREAMING PIPELINE USING APACHE KAFKA

AIM

To simulate a real-time streaming pipeline using Apache Kafka by producing and consuming live temperature sensor data using Python.

PROCEDURE

1. **Set Up Kafka Environment:**

    - Install and configure **Apache Kafka** and **Zookeeper**.
    - Start the servers:
        - `bin/zookeeper-server-start.sh config/zookeeper.properties`
        - `bin/kafka-server-start.sh config/server.properties`

2. **Create a Kafka Topic:**

    - Create a topic to stream temperature data:
    - `bin/kafka-topics.sh --create --topic temperature-readings --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1`

3. **Install Required Python Libraries:**

    - Install `kafka-python` via pip:
    - `pip install kafka-python`

4. **Write and Run the Producer Script:**

    - Use `temperature_producer.py` to simulate sensor data.
    - Sends a new reading (JSON format) every 2 seconds.

5. **Write and Run the Consumer Script:**

    - Use `temperature_consumer.py` to read messages in real-time.
    - Applies simple logic to flag high or low temperature readings.

6. **Observe Output:**

    - Monitor the consumer terminal for incoming messages and alerts based on temperature thresholds.

PROGRAM

Python Kafka Producer: temperature_producer.py

```
from kafka import KafkaProducer

from datetime import datetime

import time

import random

import json

import logging

# Configure logging

logging.basicConfig(level=logging.INFO)

logger = logging.getLogger("KafkaProducer")


# Initialize Kafka Producer

producer = KafkaProducer(

    bootstrap_servers='localhost:9092',

    value_serializer=lambda v: json.dumps(v).encode('utf-8')

)

def generate_sensor_data(sensor_id):

    """

    Simulates temperature readings from a sensor

    """

    return {

        "sensor_id": sensor_id,

        "timestamp": datetime.utcnow().isoformat(),
```

```
        "temperature": round(random.uniform(20.0, 35.0), 2)
    }


if __name__ == "__main__":
    topic = "temperature-readings"
    sensor_id = "sensor_001"


    try:
        while True:
            data = generate_sensor_data(sensor_id)
            producer.send(topic, value=data)
            logger.info(f"Sent data: {data}")
            time.sleep(2)  # Send a reading every 2 seconds
    except KeyboardInterrupt:
        logger.info("Producer stopped.")
    finally:
        producer.close()
```

Python Kafka Consumer: temperature_consumer.py

```
from kafka import KafkaConsumer
import json
import logging
# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("KafkaConsumer")
```

```
# Initialize Kafka Consumer
consumer = KafkaConsumer(
    'temperature-readings',
    bootstrap_servers='localhost:9092',
    auto_offset_reset='earliest',
    enable_auto_commit=True,
    group_id='temperature-monitor-group',
    value_serializer=lambda v: json.loads(v.decode('utf-8')),
    value_deserializer=lambda v: json.loads(v.decode('utf-8'))
)
logger.info("Started temperature consumer...")
try:
    for message in consumer:
        data = message.value
        temp = data['temperature']
        status = "Normal"
        if temp > 30:
            status = "□ High Temp Alert"
        elif temp < 22:
            status = "□ Low Temp Warning"

        logger.info(f"[{data['timestamp']}] Sensor: {data['sensor_id']} | Temp: {temp}°C | Status: {status}")
except KeyboardInterrupt:
```

```
    logger.info("Consumer stopped.")

finally:

    consumer.close()
```

OUTPUT

```
roducer Output (temperature_producer.py)
NFO:KfkaProducer:Sent data: 'sensor_id': 'sensor_ _001',
timestamp': '2025-04-05T12:10:03.123456', 'temperature: 24
NFO:KfkaProducer:Sent data: 'sensor_id': 'sensor_ _001',
timestamp': '2025-04-05T12:10:07.345678', 'temperature: 32
NFO:KfkaProducer:Sent data: 'sensor_id': 'sensor_ _001',
timestamp': '2025-04-05T12:10:09.456789', 'temperature: 27

onsumer  Output (temperature_consumer.py)
NFO:KalkaConsumer:Started temperature consumer...
NFO:KalkaConsumer:[2025-04-05T12:10:03.123456] Sensor: 001
 Temp: 24.56℃ | Temp: 21.55°C | Status: Normal
NFO:KalkaConsumer:[2025-04-05T12:10:07.345678] Sensor: 001
 Temp: 32.12℃ | Temp: 32.12°C | Status: ⚠️ High Temp Al
NFO:KalkaConsumer:[2025-04-05T12:10:09.456789] Sensor: 001
 Temp: 27.34℃ | Temp: 27.34°C | Status: Normal
```

DEPARTMENT OF AI&DS		
AIM & ALGORITHM	**20**	
PROGRAM	**30**	
EXECUTION & OUTPUT	**30**	
OBSERVATION	**10**	
VIVA VOCE	**10**	
TOTAL	**100**	

RESULT

A real-time streaming pipeline was successfully simulated using Apache Kafka, with a Python producer sending live temperature data and a consumer processing it in real time.

EX.NO:6	ORCHESTRATE A WORKFLOW USING APACHE AIRFLOW

AIM

To orchestrate an ETL workflow using Apache Airflow by defining and scheduling interdependent tasks using a Directed Acyclic Graph (DAG).

PROCEDURE

1. **Install Apache Airflow:**

    - Use pip and constraints file:

    ```
    pip install apache-airflow
    airflow db init
    ```

2. **Create a DAG File:**

    - Navigate to the `dags/` folder in your Airflow home directory.
    - Create a Python file (`etl_workflow_dag.py`) with tasks for **Extract**, **Transform**, and **Load**.

3. **Define Python Tasks:**

    - Use `PythonOperator` to wrap individual functions simulating each ETL stage.

4. **Set Task Dependencies:**

    - Ensure `Extract → Transform → Load` flow using bitshift operators.

5. **Start Airflow Web Server & Scheduler:**

    - Run:

    airflow webserver --port 8080

    airflow scheduler

6. **Trigger and Monitor the DAG:**

    - Use Airflow UI at `http://localhost:8080` to trigger and monitor the DAG execution.

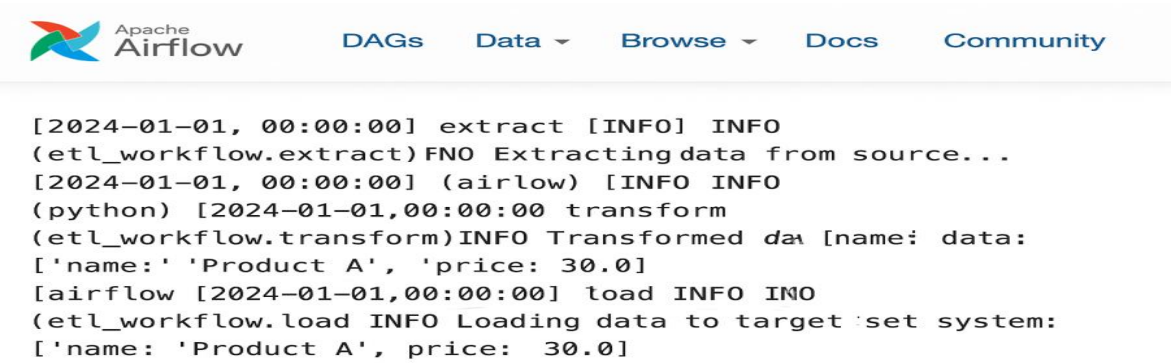
PROGRAM

(etl_workflow_dag.py)

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from datetime import datetime

import logging

def extract():

    logging.info("Extracting data from source...")

    data = {"name": "Product A", "price": 25}

    return data

def transform(**context):

    data = context['ti'].xcom_pull(task_ids='extract')

    data["price"] *= 1.2  # Add 20% tax

    logging.info(f"Transformed data: {data}")

    return data

def load(**context):

    data = context['ti'].xcom_pull(task_ids='transform')

    logging.info(f"Loading data to target system: {data}")

default_args = {

    'start_date': datetime(2024, 1, 1),

    'owner': 'airflow',

    'retries': 1

}

with DAG(dag_id='etl_workflow',
```

```
        default_args=default_args,

        schedule_interval='@daily',

        catchup=False) as dag:

    t1 = PythonOperator(task_id='extract', python_callable=extract)

    t2 = PythonOperator(task_id='transform', python_callable=transform, provide_context=True)

    t3 = PythonOperator(task_id='load', python_callable=load, provide_context=True)



    t1 >> t2 >> t3
```

OUTPUT



DEPARTMENT OF AI&DS		
AIM & ALGORITHM	**20**	
PROGRAM	**30**	
EXECUTION & OUTPUT	**30**	
OBSERVATION	**10**	
VIVA VOCE	**10**	
TOTAL	**100**	

RESULT

A DAG was successfully created and executed using Apache Airflow to orchestrate an ETL workflow involving extract, transform, and load tasks.